Voxelforge - Development Roadmap & Milestone Tracker

> **Instructions for Claude Code:**
>
> 1. Always check this file to identify the active phase and current task before writing code.
> 2. Follow all architectural guardrails in `CLAUDE.md` (Host-Authority, No Singletons, Message-Driven Voxels).
> 3. When a milestone task is implemented and verified by the user, update `[ ]` to `[x]` in this file.

Phase 1: The Voxel Foundation ("Hello, World of Blocks")

Goal: Generate a high-performance, 3D blocky terrain with modern lighting and PBR textures.

- [x] **1.1 Engine & MCP Setup:** Initialize repository, configure `.gitignore`, and verify Engine MCP + Blender MCP connections.
- [x] **1.2 Chunk Data Architecture:** Create `VoxelChunk` data structures (`16x16x16` or `32x32x32`) storing Block ID, Durability, and Light/State bytes. *(Built as 32^3 with 4-byte voxels; max durability lives on the block type and damage is stored sparsely. See DECISIONS.md D-003.)*
- [x] **1.3 Greedy Meshing & Culling:** Implement background-threaded Greedy Meshing and hidden-face culling so chunks render at 120+ FPS. *(Burst gather + greedy mesh jobs, 140–158 FPS on a 16×4×16-chunk test world. See DECISIONS.md D-004.)*
- [x] **1.4 Procedural Noise Terrain:** Generate rolling hills, caves, and biomes from a deterministic `WorldSeed` integer. *(4 WoW-zone-style biomes, tunnels + caverns, ore clusters; 2,048 chunks in ~1 s at ~122 FPS. See DECISIONS.md D-005.)*
- [x] **1.5 Art & Shaders (Creative):** Create PBR Triplanar/Block materials (Grass, Dirt, Stone, Mana Crystal Ore) with Normal/Height maps and dynamic lighting. *(Blender-baked painted textures for all 8 blocks in Texture2DArrays, URP PBR shader with parallax, Forward+ lights and glowing ore; ~120 FPS. See DECISIONS.md D-006.)*

Phase 2: Dynamic Couch Co-Op & Smooth Heroes

Goal: Get two smooth RPG characters running around the blocky world together on one PC.

- [x] **2.1 Local Listen-Server Setup:** Configure game to launch in local `Host Mode` (Server + 2 Local Clients). *(One host connection owns both couch players, told apart by local slot; server-authoritative input RPCs and voxel box collision. See DECISIONS.md D-007.)*
- [x] **2.2 Dual Input Binding:** Map Player 1 (Keyboard/Mouse or Gamepad 1) and Player 2 (Gamepad 2) via native local player subsystems. *(Press-to-join: each player gets an `InputUser` paired to their own devices; disconnect/reconnect and controller takeover; split-keyboard dev toggle. See DECISIONS.md D-008.)*
- [x] **2.3 Voronoi Dynamic Split-Screen:** Build midpoint shared camera when players are `< 15m` apart; smoothly split via angled screen-space render targets when `> 15m` apart. *(Seamless split at 15 m, free look per player when apart, camera-relative movement, 3–4 player grid. Split is 109 FPS in the editor vs 183 merged; reaching 120+ moves to 2.4. See DECISIONS.md D-009.)*

- ☑ **2.4 Multi-Viewer Chunk Loader:** Update Chunk Manager to stream/unload terrain around a `List<ViewerTransform>` so both players can explore independently. Draw chunks with Unity 6's GPU Resident Drawer + GPU occlusion culling (block shader needs DOTS instancing) so each extra split-screen camera is cheap: split-screen must reach 120+ FPS (see D-009). *(Streams whole chunk columns in a 256 m circle around every player, with a 2-chunk unload margin, pooled chunk objects and edge fog; GPU Resident Drawer + GPU occlusion culling with a DOTS-instanced block shader. Play-tested in split-screen. See DECISIONS.md D-010.)*
- ☑ **2.5 Smooth Character Integration (Creative):** Import stylized 3D characters with Mixamo animations, 1m auto-stepping, and IK foot placement on blocky stairs. *(Our own hero "the Wayfarer", built in Blender by script and rigged by Mixamo through a closed-shell stand-in; team-colour cloth per player; auto-step up and down 1-block steps, heroes face where they walk, foot IK that keeps feet on block edges, camera collision with terrain. Play-tested. See DECISIONS.md D-011.)*
- ☑ **2.6 Foliage & Decoration Layer (Creative):** Add a non-voxel decoration layer of stylized, hand-painted (WoW-*inspired*, original or CC0) grass tufts, flowers, bushes and trees on top of the blocky terrain. Place decorations deterministically from `WorldSeed` + block position during chunk meshing (off the main thread), draw them with GPU instancing plus a wind-sway shader and distance fade, and stream them with the multi-viewer chunk loader (2.4). Decorations are rebuilt whenever their supporting voxel changes, so they need no save data or network sync. Trees are pretty meshes that fill invisible `TreeTrunk` voxels, so they block building, give collision, and are chopped through `RequestModifyVoxel` (dropping wood blocks once 3.2 lands). Add per-biome color tinting and voxel corner ambient occlusion. FPS must hold at 120+ in split-screen. *(Seeded trees over invisible solid TreeTrunk voxels; grass, flowers and bushes placed by a Burst job beside greedy meshing; per-camera GPU culling with RenderMeshIndirect, wind sway and dithered fade; 10 Blender-built plants with LODs on one painted atlas; per-biome grass/leaf tint on the green parts only; corner AO from a 26-neighbour gather. Split-screen 124-139 FPS. Play-tested. See DECISIONS.md D-012.)*
- ☑ **2.7 Playable Demo Build:** Build a runnable Windows `.exe` (into a git-ignored `Builds/` folder) and play-test it outside the editor: both players exploring the world in split-screen. Check that it starts cleanly, that terrain, textures, foliage, lighting and glowing ore look the same as in the editor, and that FPS holds at 120+. Record the results (FPS, build size, issues found) in `DEVLOG.md`. *(Menu-driven `DemoBuilder` makes a 110 MB Mono/D3D12 build into `Builds/Windows/`; it starts cleanly and matches the editor. Split-screen averages 153 FPS (87 FPS 1% low), one player 252. F4 records FPS to `frame-stats.txt`, F12 saves screenshots, Shift+Esc quits, and `-smoketest` checks a build on its own. Play-tested. See DECISIONS.md D-013.)*

Phase 3: "Forge & Carve" (Digging, Building & Saving)

Goal: Let players mine tunnels, build castles, and save their world in a network-ready format.

- ☑ **3.1 Message-Driven Voxel Edits:** Implement `RequestModifyVoxel(Coord, BlockID, InstigatorID)`. Never mutate chunk arrays directly from client input. *(One public entry point with checked results; a net-change `VoxelDeltaLog` that edits replay from when chunks generate again; owner-checked server RPC with an 8 m reach limit; chopping a trunk fells the whole tree; temporary dig/place controls that aim with the camera and show a team-coloured target outline. Play-tested in split-screen. See DECISIONS.md D-014.)*
- ☑ **3.2 Block Durability & Mining:** Add raycast/spherecast targeting, block cracking decals, and block item drops. Store in-progress mining damage in a sparse map (see D-003). *(Hold Mine and the host aims from the controls it received and wears the block down in a sparse damage map that heals after 3 s;*

8 crack stages; drops that hop, merge and fly into a per-player block bag; a new Log block from chopped trees; a team-coloured reticle and bag readout per split-screen view. Spherecast waits for 4.x combat targets. Play-tested in split-screen. See DECISIONS.md D-015.)

- [x] **3.3 Grid Snap Placement:** Add ghost-block preview and placement validation (prevent placing blocks inside characters). *(Place builds with the selected block from your bag; the host aims from the controls it received and checks the snapped cell against every player with `VoxelPlacement`, refunding a failed edit. A see-through ghost block in your team colour turns red over a character, hides while mining and toggles per player; pick blocks with the wheel, LB/RB or Z/C and ./ keys; hold to build walls; pillar-jumping works. Play-tested in split-screen at 166 FPS shared camera. See DECISIONS.md D-016.)*
- [x] **3.4 Seed + Delta Save System:** Save world using only `WorldSeed` + a compressed dictionary of modified chunk deltas and `PlayerUUID` profiles. *(One GZip file per world: seed, generator version, net block deltas (current + generated, so saves notice a changed generator) and a profile per `PlayerUuid` with position, facing and block bag. Each PC remembers a UUID per local slot, so couch players get their own profile back in any join order. Saves every 2 min, on quit and on F5; writes on a worker with a `.bak` backup. 10,000 edits = 865 bytes. Also: the camera can now tilt up, so you can mine and build above your head. Play-tested in split-screen. See DECISIONS.md D-017.)*
- [x] **3.5 Props & Tools (Creative):** Model 3D pickaxes, torches (with real point lights), and interactive workbenches. Add items beyond blocks: a data-driven `ItemDefinition` with permanent `ItemId`s (blocks keep their IDs, other items start at 1000), with the bag holding `ItemStack`s (same save format: a slot was already an ID plus a count). Each block gets a harvest kind (Pick / Axe / Any), and the held tool multiplies mining power for that kind, read by the host. Tools: Wooden Pickaxe, Stone Pickaxe, Stone Axe, and Kian's **The Pick of Destiny** (an axe on one side and a pick on the other, the best tool for both mining and chopping, crafted from a Stone Pickaxe + Stone Axe + Mana Crystal Ore). Held items show in the hero's right hand on every PC. Torches are a non-solid block that stands on the floor or hangs on a wall, with a pool of flickering URP point lights given to the torches nearest each viewer. The Workbench is a placeable block with a prop model: an Interact button opens a recipe list in that player's own viewport, and the host checks reach and bag contents on `RequestCraftRpc` (a minimal `RecipeDefinition` that 5.6 extends). Also model the M.G.R.S. for 3.6. FPS must hold at 120+ in split-screen. *(Tools, torches and a workbench as prop blocks drawn by the decoration layer; the host checks the held tool from the bag and multiplies mining by harvest kind (The Pick of Destiny: 5x, stone in 0.3 s); a crafting window per split-screen view; a pool of 32 flickering torch lights; every model and texture drawn by `build_props.py`. Play-tested in split-screen. See DECISIONS.md D-018.)*
- [ ] **3.6 Kian's M.G.R.S. (Magic Gun Rifle Staff) (Creative):** A wizard's staff that also shoots out the end. Craft it at the workbench and hold it like a tool. Firing it launches a glowing *Mana Bolt* projectile, moved by the host, that damages the block it hits through `RequestDamageVoxel` (longer reach than mining, with a cooldown), with a muzzle flash, a light that travels with the bolt, and an impact burst. In Phase 4 it becomes a real weapon on the 4.1 ability framework (mana cost, damage to creatures).
- [ ] **3.7 Split-Screen Spatial Audio & Voxel Foley:** Build a multi-viewer spatial audio router (attenuating by the closest local viewer in `List<ViewerTransform>` from 2.4 and stereo-panning per split-screen viewport without single-player `AudioListener` shortcuts) and data-driven per-block surface audio profiles. Add footsteps and jump/land sounds driven by the voxel under each foot (2.5), mining hit sounds that pitch up across the 8 crack stages (3.2), block break and place SFX (3.1, 3.3), Mana Crystal Ore harmonic chimes, tree-felling creaks + leaf rustles, item drop pickup pops, and looping torch crackles (3.5), and the M.G.R.S. Mana Bolt firing and impact (3.6).
- [ ] **3.8 Playable Demo Build:** Build a new Windows `.exe` (version `0.3.x`, same `DemoBuilder` as 2.7) and play-test the whole Forge & Carve loop outside the editor: both players in split-screen mining

tunnels, chopping trees, picking up block drops, building with the ghost-block preview, placing torches and a workbench, crafting tools up to The Pick of Destiny, blasting blocks with the M.G.R.S., and hearing footsteps, mining hits, block breaks, and torch crackles clearly across both split-screen viewports. Then quit, restart, and check that every edit and both player profiles come back from the save. Check that digging and building remesh without visible hitches (both players editing at once), that torch lights hold up in tunnels, and that FPS holds at 120+ in split-screen. Record the results (FPS and 1% low, save file size, load time, build size, issues found) in `DEVLOG.md`.

Phase 4: The Warcraft Injection (Classes, Races, Spells & RPG UI)

Goal: Layer *World of Warcraft* combat, mana, cooldowns, and talent trees onto the voxel world.

- ☐ **4.1 Data-Driven Ability Framework:** Build ScriptableObject/DataAsset spell templates (Mana Cost, Cooldown, Cast Time, Element, Range, AoE Radius).
- ☐ **4.2 Spell Logic vs. Visual Cue Split:** Separate authoritative damage/terrain destruction math from client-side particle VFX and audio.
- ☐ **4.3 Destructive Spell Mechanics:** Allow explosive spells (e.g., *Pyroblast*, *EarthShatter*) to batch-modify voxel chunks and carve real craters.
- ☐ **4.4 Per-Player Split-Screen HUD:** Build independent Action Bars (1-9 hotkeys/d-pad), cast bars, and floating combat text confined to each player's viewport.
- ☐ **4.5 Generous Bag & Visual Inventory System:** Expand the 3.5 item bag into a generous, frustration-free RPG inventory window inside each player's viewport (4.4): auto-sorted category tabs (*Blocks*, *Gear & Tools*, *Consumables*, *Materials*) with a large starting capacity, `999` stack sizes for blocks and materials so building never crowds out RPG loot, and equippable bag slots that expand capacity even further. Add one-button Auto-Sort and a server-authoritative **"Send to Partner"** action so couch co-op players can toss items directly to each other. Model a **3D wearable backpack/satchel** on each hero's back that visually reflects their equipped bag, a **3D drawstring loot bag** prop for dropped items, and a placeable **Voxel Storage Chest** block (persisted in 3.4 world saves) for stashing items at a base.
- ☐ **4.6 Class & Spell Design (Creative):** Design first 3 playable classes (*Runeforged Paladin*, *Geomancer Mage*, *Shaman*) with 3 signature abilities each. The Shaman plants elemental totems on blocks (server-owned entities that buff, heal or attack nearby) and has at least one earth spell that changes terrain through `RequestModifyVoxel` (e.g. *Earthen Wall* raises a real block wall).
- ☐ **4.7 Playable Races & the Thunderhoof (Creative):** Add a data-driven `RaceDefinition` (model, stats, racial passive, racial ability) and a race choice on character creation. First races: the *Thunderhoof*, our own big, gentle bull-folk of the plains with a deep shaman tradition (original name, look and lore, no Blizzard IP), plus one more race. Thunderhoof racials: *Hoofquake* (short AoE stun around you) and *Herb-Sense* (bonus Herbalism yield, 5.5). Add a *Thunderhoof Plains* homeland biome (windswept grassland, mesas, tall grass) to the seeded terrain. Model the Thunderhoof in Blender with a larger humanoid rig so it can share the 2.5 Mixamo animations.

Phase 5: Monsters, Dungeons & Bosses

Goal: Populate the world with intelligent RPG creatures that can navigate dynamic blocky terrain, and close the gather → craft → gear-up loop (ores, herbs, mob drops → potions, armor, weapons).

- ☐ **5.1 Dynamic Voxel NavMesh:** Configure runtime NavMesh rebuilding so monsters recalculate paths when players dig trenches or build walls.

- [ ] **5.2 RPG Threat & Aggro System:** Implement a *WoW*-style threat table so tank characters can hold boss aggro while ranged characters cast spells.
- [ ] **5.3 Procedural Dungeon Stamps:** Stamp pre-built boss arenas and ruins into the procedural voxel world during chunk generation.
- [ ] **5.4 Loot & Equipment System:** Implement armor slots, weapon attachment sockets, and rarity tiers (Common, Rare, Epic, Legendary). Mobs and bosses use data-driven loot tables, rolled on the server, that drop both gear and crafting materials (hides, fangs, elemental essences) into each player's 4.5 bag. Each couch player gets their own loot so co-op partners never fight over drops.
- [ ] **5.5 Gathering Professions:** Add *Mining* (ore blocks from 3.2 drop ore items; rarer ore tiers by depth and biome), *Herbalism* (special herb nodes from the 2.6 foliage layer that can be harvested, placed by biome, and respawn on a timer kept in a sparse server-side map like D-003's mining damage), and *Skinning/Salvaging* (gather extra materials from defeated mobs). Each profession has a skill level that rises with use and unlocks higher-tier nodes. Every harvest is a server-authoritative request carrying the `InstigatorPlayerID`.
- [ ] **5.6 Crafting Professions:** Add data-driven `RecipeDefinition` assets (inputs, output, required station and skill level) for *Alchemy* (health/mana potions and elixirs, used through the 4.1 ability framework), *Blacksmithing* (plate armor and weapons from ore), and *Leatherworking* (leather armor and upgradeable 4.5 backpacks/satchels from hides). Crafting happens at placeable voxel stations (Alchemy Table, Forge + Anvil, Tanning Rack, extending the 3.5 workbench). Each player has their own split-screen crafting window (4.4). Crafted gear plugs into the 5.4 slots and rarity tiers.
- [ ] **5.7 Monster, Boss & Crafting Art (Creative):** Model and animate 3 overland mobs and 1 multi-phase Dungeon Boss with telegraph attacks. Create 3+ herb models, 3 ore tiers, crafting-station props, potion bottles, and one craftable armor set + weapon per class.

Phase 6: The Tavern Door (Front-End, Loading Screen & Settings)

Goal: Build a polished front-end flow, an asynchronous chunk-streaming loading screen, and scalable video/display settings before opening the game to online PCs.

- [ ] **6.1 Title Screen, World Picker & Character Creation Flow:** Build the front-end state machine transitioning from Title Screen → World Selection (create new world with `WorldSeed` or load existing save from 3.4) → Per-Player Couch Co-Op Lobby & Character Creation (each local player joins on their own `InputUser` device from 2.2 and selects or creates a profile with Race from 4.7 and Class from 4.6).
- [ ] **6.2 Async Loading Screen & Chunk Readiness Gate:** Implement a loading screen between front-end launch and active gameplay that reports world initialization, initial chunk generation and greedy meshing progress around all viewer positions (2.4), and network delta sync progress (7.2), holding player physics until starting terrain colliders are ready.
- [ ] **6.3 Screen, Display & Audio Settings:** Build persistent machine-level display and audio settings covering Resolution, Window Mode (Exclusive Fullscreen, Borderless Window, Windowed), Monitor selection, VSync, Frame Rate Cap, UI Scale, Split-Screen camera preferences (such as the Voronoi split distance threshold from 2.3), and Master / SFX / Ambience / Music volume sliders (plus Mute on Focus Loss) wired into the 3.6 audio router.
- [ ] **6.4 Video Quality & Voxel Performance Settings:** Add graphics quality presets (Low / Medium / High / Ultra) and granular sliders/toggles wired into URP and the voxel engine: Chunk Streaming Distance (2.4), Foliage Density & Draw Distance (2.6 `RenderMeshIndirect`), URP Render Scale, Shadow Distance & Cascades, Anti-Aliasing, Parallax Mapping (1.5), and Voxel Corner AO (2.6).

- [ ] **6.5 In-Game Pause & Settings Menu:** Add a per-player-aware Pause/Options overlay accessible in-game (via the initiating `InputUser` without single-player shortcuts) to adjust Screen, Audio & Video Quality settings live, trigger a manual save (3.4), or cleanly shut down the local Listen-Server and return to the Title Screen.
- [ ] **6.6 Title Diorama, Loading Art & Menu Framing (Creative):** Create a live 3D voxel diorama backdrop for the Title and Character Creation screens (a stylized camp/tavern scene with swaying foliage, torchlight, and glowing ore), stylized RPG menu frames, loading bar art with gameplay tips, and menu/loading ambient audio.
- [ ] **6.7 Playable Demo Build:** Build a new Windows `.exe` (version `0.6.x`, same `DemoBuilder` as 2.7) and play-test the full front-end-to-gameplay loop outside the editor: booting to the Title Screen, changing window mode/resolution, audio volume sliders, and video quality presets, creating a world and two heroes, transitioning through the loading screen into split-screen gameplay, tweaking chunk/foliage/audio settings live from the Pause Menu, and returning to Title to reload the saved world. Verify 120+ FPS at default quality settings and record results in `DEVLOG.md`.

Phase 7: Opening the Portal (Internet Multiplayer)

Goal: Enable online co-op with friends while preserving local Dynamic Split-Screen.

- [ ] **7.1 Relay Transport Swap:** Integrate Steamworks P2P or Epic Online Services (EOS) transport layer (no port-forwarding required) and wire Host/Join lobby flows into the 6.1 Title Screen.
- [ ] **7.2 Initial World Sync:** Send `WorldSeed` + compressed `VoxelDeltaLog` to joining remote players upon connection, feeding sync progress into the 6.2 Loading Screen before spawning.
- [ ] **7.3 Bandwidth Optimization:** Compress chunk delta broadcasts and test mixed mode (2 local split-screen players + 1-2 remote online players).